

Emporika: AI Chatbot for PrestaShop

The [Emporika](#) AI chatbot for PrestaShop 8.1+ and 9 is an online salesperson that closes sales in your store: shoppers describe what they need or upload a photo of something they like, it finds matching products from your catalog, handles objections like "too expensive" with alternatives instead of a discount, answers questions from your CMS pages, and walks shoppers to cart and checkout in 65+ languages. This module syncs your product catalog and CMS pages to Emporika, embeds the chat widget on your storefront, and answers the chat's cart and order status requests.

[\[image\]](#)

- **Integration overview:** emporika.com/integrations/prestashop/
- **Full documentation:** emporika.com/docs/prestashop/ (webhook format reference, hook examples, troubleshooting)
- **Features:** emporika.com/features/ · **FAQ:** emporika.com/faq/ · **Pricing:** emporika.com/pricing/
- **Live demo:** demo.emporika.com and a [30-second video](#). That demo sells electronics, and the behavior is the same on any catalog.

Requirements

- PrestaShop 8.1+ or 9.x (tested on 8.1 to 9.2)
 - PHP 8.0+ on PrestaShop 8.1, PHP 8.1+ on PrestaShop 9. PrestaShop 8.1 also runs on PHP 7.x; this module does not
 - An [Emporika account](#). Sign up with no card; \$25 of signup credit (~100 free conversations) auto-applied
-

Installation

1. Get the module from the [PrestaShop Addons Marketplace](#) (paid) or from [GitHub](#) (free).
2. In your PrestaShop back office, go to **Modules > Module Manager > Upload a Module** and upload `emporiqa.zip`.
3. Click **Configure** on the Emporiqa module.
4. Click **Connect to Emporiqa**. A new tab opens on [emporiqa.com](#). Create a free account (no card required, \$25 of signup credit) or sign in if you already have one, then pick the store you want to connect (or create a new one). The module is connected when you return.
5. On the **Sync** tab, click **Send my catalog** and keep the tab open until the progress bar is full. Products, pages, and combinations flow through; the widget appears on your storefront when the first product arrives.

On HTTP, or prefer to paste credentials yourself? Expand **Edit credentials manually** on the Configure page. Paste a **Store ID** and **Connection Secret** from your Emporiqa dashboard under **Settings → Integration**. Both flows reach the same place.

For order status in the chat, copy the **Order tracking** address from the Configure page into your Emporiqa dashboard under **Settings → Integration → Order tracking**. It is on by default. Once Emporiqa has told the module that your store has ready-made rules, the Configure page shows a **Ready-made rules** section instead, with each rule marked **On** or **Not added** and an **Open in Emporiqa** link; there the **Order status** rule replaces the order tracking address, which moves under **Advanced**. The module learns whether your store has ready-made rules when you connect and each time you click **Test Connection**.

Configuration

All settings are managed from the module configuration page (**Modules > Emporiqa > Configure**):

Connection Settings

The recommended path is **Connect to Emporiqa** (one-click handshake, no credentials to paste). For HTTP sites or manual setup, expand **Edit credentials manually**:

Setting	Description	Default
Store ID	Your Emporiqa store identifier (filled automatically by one-click connect)	(none)
Connection Secret	HMAC-SHA256 signing secret (filled automatically by one-click connect)	(none)

Shops and languages

Setting	Description	Default
Shops (multistore only)	Shops whose catalog is synced and where the chat is shown. All shops share one Emporiqa store; each shop is a channel	All active shops
Languages	Languages included in sync payloads; their pages show the chat. Pages in an unticked language show no chat	All active shop languages

Order tracking

Setting	Description	Default
Order tracking	Lets the chat answer "Where is my order?" after the shopper gives the order reference and the order's email. Shown in its own section, or under Advanced as <i>Old order tracking</i> once ready-made rules are offered to your store, where the Order status rule replaces it	On

Advanced

Setting	Description	Default
Auto-sync products	Send each product change as it is saved	On
Auto-sync pages	Send each CMS page change as it is saved	On
Webhook URL	Emporiqa webhook endpoint	<code>https://emporiqa.com/webhooks/sync/</code>
Batch Size	Products/pages per webhook request during bulk sync	25

In-chat cart operations are always enabled. No configuration needed.

AI disclosure

The chat's default greeting tells the shopper it is the store's AI assistant, in every language the chat speaks. A custom greeting must keep that disclosure; one that drops it is refused when you save it. Section 8.6 of the [Emporiqa Terms](#) treats removing the disclosure, including through custom CSS or custom code, as a breach.

Keeping your catalog in sync

The module pushes product, page, and order changes to Emporiqa automatically as they happen via PrestaShop hooks. Per-product changes such as scheduled promos (SpecificPrice), image edits, and combination edits re-emit the affected product on their own; pure stock/out-of-stock changes emit a compact availability-only update instead of rebuilding the whole product.

Some changes affect the whole catalog (category or brand renames, currency rate refreshes, tax-rate or tax-rules-group edits, cart-rule

changes, new languages enabled). Running a synchronous per-product re-sync from those hooks would block the admin request, so the module logs an actionable warning in **Advanced Parameters** → **Logs** instead and leaves the catalog refresh to a manual run.

Re-run a full sync from the **Sync** tab when:

- You see one of the "catalog-wide change" warnings in the PrestaShop log
- You add a new shop in multi-shop mode (existing products won't carry the new shop's data until something else touches them)
- You import products in bulk from a CSV file (PrestaShop sometimes bypasses standard save hooks during bulk imports)
- A custom script, migration, or another module writes catalog data directly to the database
- Emporiqa was unreachable for an extended period (network outage, planned maintenance, expired credentials)

As a safety net, run a full sync once a week to catch any drift that may have built up from background failures.

Product payload fields

Beyond the fields shown in the [webhook payload reference](#), the full product and combination payload carries these PrestaShop-native merchandising and pricing fields:

- `condition`: string or null; PrestaShop's product condition ("new", "used", or "refurbished").
- `is_virtual`: boolean; true for digital products with no shipping.
- `available_for_order`: boolean; false for display-only / catalog-mode products. The assistant still describes these but won't add them to the cart.
- `max_order_quantities`: per-channel dict (`{channel: int | null}`) of the maximum allowed per-order quantity. PrestaShop has no native per-order maximum, so this currently always ships `null` (no limit). The field is included for cross-platform contract parity, so a future custom source can populate it.

- `tier_prices`: per-currency list of quantity-based volume discounts (`[{min_quantity, price}]`), present on a price entry only when the product or combination has PrestaShop quantity discounts configured. Each tier reflects the public (guest) shopper's unit price at that break. Group-, customer-, or country-restricted (B2B) tiers are intentionally excluded.

These flags are part of the full product and combination payload, not the lightweight `product.availability` event, which carries only the identification number, SKU, per-channel availability statuses, and stock quantities.

Module structure

```
emporiqa/
├── emporiqa.php                # Main module class (hook)
├── config.xml                  # Module metadata
├── logo.png                    # Module icon
├── classes/
│   ├── EmporiqaActionEndpoint.php    # Ready-made rules
│   ├── EmporiqaCartApiEndpoint.php   # Cart API body (POST)
│   ├── EmporiqaCartHandler.php        # In-chat cart operations
│   ├── EmporiqaChannelResolver.php    # Multi-shop → channel mapping
│   ├── EmporiqaConnectHandshake.php  # One-click connect handshake
│   ├── EmporiqaConnectNonce.php       # One-click connect nonce
│   ├── EmporiqaJsonResponse.php       # The one JSON response
│   ├── EmporiqaLanguageHelper.php     # Language mapping
│   ├── EmporiqaOrderFormatter.php     # Order payload formatting
│   ├── EmporiqaOrderStatus.php        # Order status lookup
│   ├── EmporiqaOrderTrackingEndpoint.php # Order tracking endpoint
│   ├── EmporiqaPageFormatter.php      # CMS page payload formatting
│   ├── EmporiqaProductFormatter.php   # Product/combination formatting
│   ├── EmporiqaSchema.php             # Module tables, constraints
│   ├── EmporiqaSignatureHelper.php    # HMAC-SHA256 signing
│   ├── EmporiqaSyncService.php        # Bulk sync orchestration
│   ├── EmporiqaTokenEndpoint.php      # Signed customer token
│   └── EmporiqaWebhookClient.php      # HTTP client for webhooks
```

```

├─ controllers/
│   └─ admin/
│       ├── AdminEmporiqaController.php          # Admin me
│       └─ AdminEmporiqaConnectController.php    # One-click
│   └─ front/
│       ├── action.php                          # Ready-made rules
│       ├── cartapi.php                         # Cart API endpoint
│       ├── ordertracking.php                   # Order tracking en
│       └─ token.php                            # Customer token en
├─ views/
│   ├── css/admin.css                          # Admin configurati
│   ├── img/                                  # Module images (re
│   ├── js/
│       ├── admin-sync.js                      # Bulk sync UI with
│       ├── front-cart-handler.js              # Chat widget cart i
│       └─ front-customer-token.js             # Hands the customer
│   └─ templates/
│       ├── admin/configure.tpl                 # Configuration pag
│       ├── admin/sync_tab.tpl                  # Sync tab template
│       ├── admin/sync_health_line.tpl          # Last full sync l
│       └─ hook/header.tpl                      # Widget embed (dis
├─ translations/fr.php                         # French back-offic
└─ upgrade/                                    # Version upgrade s

```

Registered PrestaShop hooks

Hook	Purpose
displayHeader	Embeds the chat widget on the storefront
actionProductSave	Syncs product on external system after update
actionProductDelete	Sends delete event to external system when product and its variants are deleted

<code>actionObjectCombination{Add, Update, Delete}After</code>	Syncs parent product when combination change
<code>actionObjectCms{Add, Update, Delete}After</code>	Syncs CMS page create/update/delete
<code>actionValidateOrder</code>	Captures chat session and sends <code>order.completed</code>
<code>actionOrderStatusPostUpdate</code>	Sends <code>order.completed</code> late payment cancellation
<code>actionUpdateQuantity</code>	Emits a lightweight <code>product.available</code> event when stock changes (no full product re-sync)
<code>actionProductOutOfStock</code>	Emits a <code>product.available</code> event on stock-based transitions
<code>actionObjectSpecificPrice{Add, Update, Delete}After</code>	Re-syncs the affected product on schedule, promos, per-product reductions, and based volume discounts (tier pricing)
<code>actionObjectImage{Add, Update, Delete}After</code>	Re-syncs the affected product when product images change
<code>actionObjectCategory{Update, Delete}After</code>	Logs an actionable warning so the merchant can run a full sync (catalog-wide images)
<code>actionObjectManufacturer{Update, Delete}After</code>	Logs an actionable warning so the merchant can run a full sync (catalog-wide images)

<code>actionObjectCartRule{Add, Update, Delete}After</code>	Logs an actionable warning so the merchant can run a full sync (catalog-wide im
<code>actionObjectCurrencyUpdateAfter</code>	Logs an actionable warning so the merchant can run a full sync (catalog-wide pr
<code>actionObjectTaxUpdateAfter / actionObjectTaxRulesGroupUpdateAfter</code>	Logs an actionable warning so the merchant can run a full sync (catalog-wide pr
<code>actionObjectLanguageAddAfter</code>	Logs an actionable warning so the merchant can run a full sync locale needs ba

Extensibility hooks

Developers can hook into the sync pipeline to customize payloads or cancel syncs:

Hook	Purpose	Key Parameters
<code>actionEmporikaFormatProduct</code>	Modify product/variation payload before sending	<code>&\$data</code> , <code>\$product</code> , <code>\$event_type</code>
<code>actionEmporikaFormatPage</code>	Modify page payload before sending	<code>&\$data</code> , <code>\$event_type</code>
<code>actionEmporikaFormatOrder</code>	Modify the <code>order.completed</code> event payload	<code>&\$data</code> , <code>\$event_type</code>

actionEmporiqaShouldSyncProduct	Conditionally cancel a product sync	\$product \$event_t &\$should
actionEmporiqaShouldSyncPage	Conditionally cancel a page sync	\$page, \$event_t &\$should
actionEmporiqaWidgetParams	Modify chat widget embed parameters	&\$params
actionEmporiqaOrderStatus	Modify the Order status ready-made rule's answer	&\$data, \$
actionEmporiqaOrderTracking	Modify the order tracking response	&\$data, \$

Pricing

The module is paid on PrestaShop Addons and free on [GitHub](#). The Emporiqa service itself is pay-as-you-go: \$0/month base + \$0.25/conversation, with \$25 of signup credit (about 100 conversations) and no card required at signup. After the credit, the monthly cap defaults to \$59 and you can change it from the billing dashboard. Voice mode (the shopper speaks and hears the answer read aloud) is optional and off by default: a conversation where the shopper speaks costs \$0.15 more, charged once, and counts toward that cap. Prices exclude VAT. Enterprise option for catalogs over 100,000 products. Full pricing at emporiqa.com/pricing/.

Known issues

- The messages written by the **Sync** and **Test Connection** buttons, and the one-click connect error messages, are in English only. The rest of the configuration page is available in English and French.

- The controllers under `controllers/` and `emporiqa.php` keep PHP 7 syntax (no trailing commas in calls or parameters), so a shop left on PHP 7 gets a clean error instead of a fatal one; the PHP 8 code lives in `classes/`.
- The full list, with technical detail, is in [CHANGELOG.md](#).

Support

Email support@emporiqa.com.

License

[Academic Free License 3.0 \(AFL-3.0\)](#)